And there you have it! At this stage, you should now be familiar with how to integrate your contract tests into a CI/CD pipeline. An excellent resource worth mentioning that can also help with further learning on this topic is a curated CI/CD workshop from the Pact team at https://docs.pactflow.io/docs/workshops/ci-cd.

Summary

- To get the full benefit of contract testing, the consumer and provider tests must be integrated into a CI/CD pipeline and be triggered automatically when different teams push changes.
- Using a unique identifier such as a GitHub commit ID or SHA ensures that contract versions can be identified from code changes.
- Pact provides a way to configure branches to ensure that providers verify the correct contract. The branches are typically repository branches from which the code is committed.
- If consumer and provider teams are working on the same feature, it's a recommended practice to use the same branch name.
- Pact recommends creating environments similar to your deployment structure to make sure that Pact can track which versions will be deployed to which environments.
- After changes are deployed to an environment, it's recommended to record the deployment or release so Pact knows which versions are deployed.
- Deployed versions in Pact work slightly different than released versions. If a version is said to be deployed in Pact, this means that previous versions will be marked as not deployed. If a version is said to be released in Pact, previous versions will still be marked as released.
- The can-i-deploy tool is a feature introduced by Pact that lets you query the matrix table to verify if certain versions ready to be deployed to an environment have a successful verification result with the version already deployed in the same environment.
- Pact webhooks need to be created and configured automatically to trigger the provider verification job when there are contract changes from the consumer.